ADR 001: Distributed Authorization, Token Delegation, and Real-Time Revocation Engine

Status: Proposed

Deciders: Software Architecture Team, Security Architecture Team

Date: August 23, 2026

---

1. Context, Assumptions & Architectural Restrictions

Caseware Collaborate requires a high-throughput, low-latency authorization framework supporting internal firm staff and external client stakeholders across organizational boundaries. The legacy authorization system has reached its infrastructure capacity limits. This design replaces it with a scalable, highly reliable architecture capable of processing current traffic (< 1 Billion requests/month) with zero explicit budget constraints.

Key System Assumptions

Inter-Service Networking: Infrastructure leverages **AWS VPC Lattice** for service-to-service communication rules and traffic management, alongside CloudMap.

Automation: CI/CD pipelines are fully automated to self-register CloudMap services and apply VPC Lattice target group routing policies.

Resource Granularity: Edge authorization acts as a coarse perimeter gate and role injector (`context.roles`), leaving resource-level enforcement (e.g., document-level overrides) to the Workspace System.

AWS API Gateway Platform Restrictions

The architecture is directly constrained by AWS API Gateway resource specifications:

1. **API Gateway Method/Route API Spec**:

   REST APIs (`AWS::ApiGateway::Method`): The `authorizationType` property accepts only a single enum value per method (`COGNITO_USER_POOLS` , `CUSTOM` , `AWS_IAM` , or `NONE`), attaching exactly one `authorizerId` .

   HTTP APIs (`AWS::ApiGatewayV2::Route`): The `AuthorizationType` property accepts a single authorizer type per route (`JWT` , `CUSTOM` , or `AWS_IAM`), attaching exactly one `AuthorizerId` .

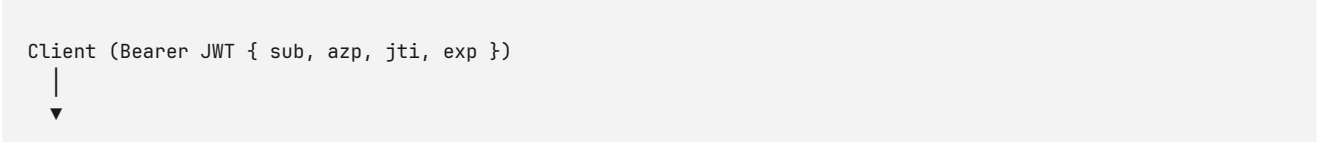
2. **Official AWS Guidance**: AWS Technical Support and AWS re:Post explicitly confirm this boundary: *"You can only have a single authorizer on every endpoint. If you need more than one mechanism, you will need to use a Lambda authorizer to verify both."*

Because Gateway routes cannot chain a native Cognito Authorizer and a custom Authorizer, token verification and authorization are consolidated into a single custom Lambda Authorizer.

---

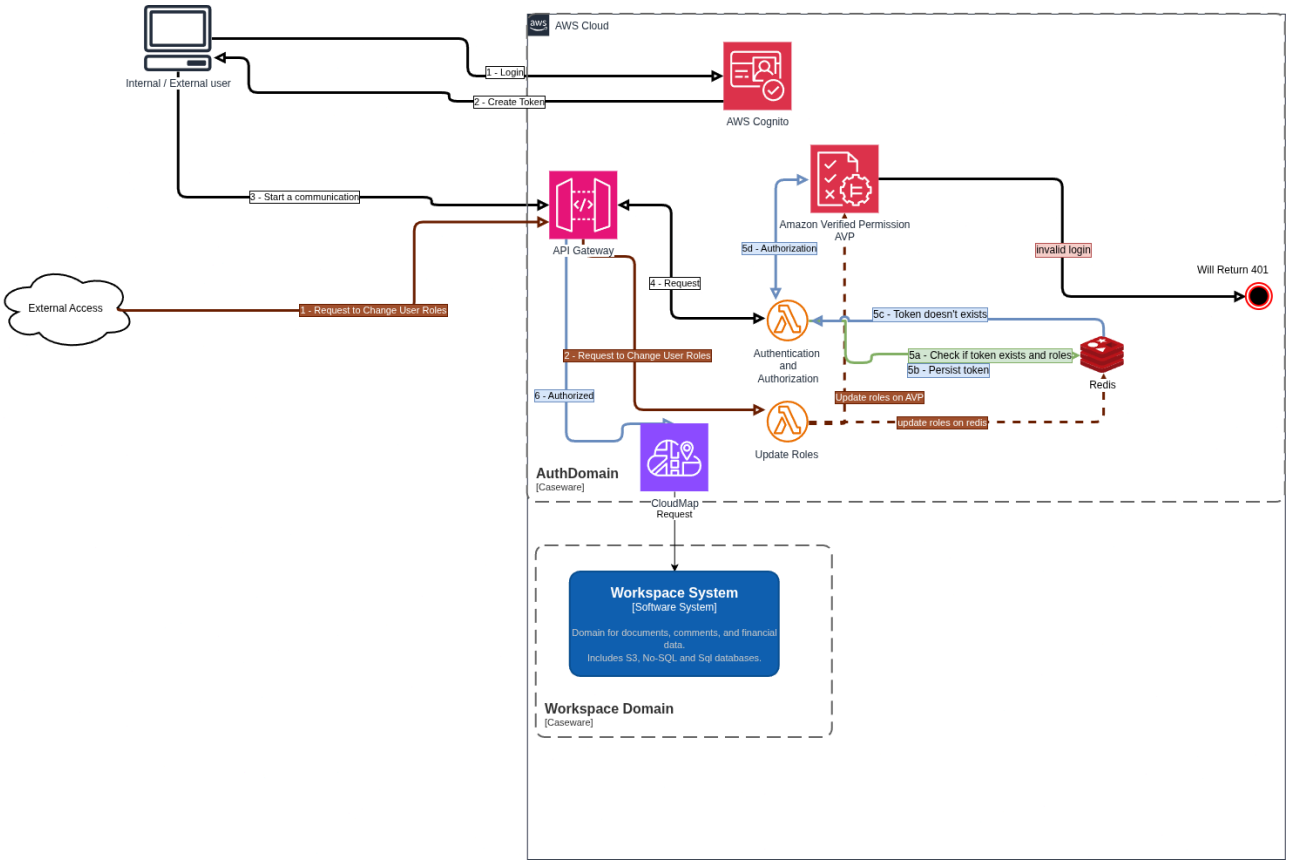
2. High-Level Architecture & Execution Flow

We will deploy a **Consolidated Custom Lambda Authorizer** attached to API Gateway HTTP API routes (`payloadFormatVersion: "2.0"`). This authorizer executes cryptographic JWT validation, Redis session lookups, and Amazon Verified Permissions (AVP) Cedar fallbacks within a single execution context before forwarding authorized requests to the Workspace System via VPC Lattice and AWS CloudMap.



```
API Gateway HTTP API (Custom Lambda Authorizer Slot — Gateway Cache TTL: 0)
  │
  ▼
┌──────────────────────────────────────────────────────┐
│  Auth Lambda Authorizer (Single Execution Context)   │
│                                                      │
│  1. Parse JWT (Zero-alloc spans) → extract kid, exp, jti │
│  2. Verify JWT signature against cached Cognito JWKS │
│  3. GET Redis key authz:{jti}                        │
│  4. HIT  → Return isAuthorized: true + context.roles │
│  5. MISS → Evaluate Cedar in AVP → SET Redis → Return v2 │
└──────────────────────────────────────────────────────┘
  │
  ├─► ALLOW  ──► API Gateway ──► VPC Link / VPC Lattice ──► Workspace System
  └─► DENY   ──► HTTP 401 / 403 Return
```

Architecture Design



Last Update
23/08/2026

Sources:
https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-oidc-idp.html
https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-identity-federation-consolidate-users.html

Cryptographic Order of Operations (Untrusted First)

To prevent unauthenticated or forged Bearer tokens from querying Redis or AVP, cryptographic validation executes before any network I/O:

1. **Token Parsing**: Parse `Authorization: Bearer <JWT>` using zero-allocation span operations to extract `kid`, `exp`, `nbf`, and `jti`.
2. **Cryptographic Validation**: Verify token signature and expiration against Cognito's public key set (`jwks.json`), cached in container memory. Trigger an HTTP JWKS fetch *only* on an unknown `kid`.

3. **Cache Lookup**: Execute `GET authz:{jti}` against Redis.

    Hit: Return `isAuthorized: true` with injected roles in `context`.

    Miss: Query AVP Cedar store for user role snapshot $\rightarrow$ Write `SET authz:{jti} <roles> EX <exp-now>` $\rightarrow$ Return `isAuthorized: true`.

Session Management & Sub-Second Revocation Integrity

Cache Keying: Sessions and role snapshots are keyed in Redis by JWT ID: `authz:{jti}` with TTL set to `exp - current_time`. Gateway cache TTL is set to `0`.

Revocation Order: To prevent a cache miss from re-hydrating stale permissions from AVP after a role change:

1. **Update Roles Lambda** writes policy changes to **Amazon Verified Permissions (AVP)** first.
2. **Update Roles Lambda** issues `DEL authz:{jti}` in **Redis** second.

On-Behalf-Of (OBO) Delegation & Confused Deputy Mitigation

External Systems: Use RFC 8693 OAuth 2.0 Token Exchange with AWS Cognito to convert client credentials + end-user consent into scoped, short-lived tokens.

Internal Microservices: Pass an explicit exchange token containing `sub` (original user), `azp` (calling service), and `aud` (target service).

Audience Enforcement: Downstream microservices strictly validate that token `aud` matches their specific service ID, blocking forwarded tokens from accessing unauthorized endpoints.

3. Implementation Plan

Phase 1: Identity & Federation: Configure AWS Cognito User Pools with per-firm OIDC/SAML federation and PKCE authorization code endpoints.

Phase 2: Policy Engine & Cache Setup: Define Cedar domain schemas in Amazon Verified Permissions and provision ElastiCache for Redis.

Phase 3: Authorizer & Networking Deployment: Deploy the custom Auth Lambda Authorizer, bind it to API Gateway HTTP API routes (`payloadFormatVersion: "2.0"`), deploy the Update Roles Lambda, and configure VPC Lattice target groups.

Phase 4: Workspace Integration: Configure CI/CD pipelines to self-register CloudMap service endpoints and enforce `aud` claim validation across microservices.

4. Testing Strategy

Cedar Policy Unit Testing: Execute offline unit tests against Cedar policy definitions to verify role hierarchy and resource override evaluations.

Load & Latency Benchmarks: Benchmark API Gateway + Auth Lambda + Redis using load testing tools targeting >30,000 requests/sec with p99 latency <10ms on hit path.

Sub-Second Revocation E2E: Automated integration tests executing live requests, triggering an administrative role update via Update Roles Lambda, and verifying that subsequent requests fail within 1 second.

Confused Deputy Pen-Testing: Security tests attempting to replay valid internal delegation tokens against mismatched downstream microservices (`aud` mismatch).

5. Evaluation & Observability

Metric Category	Metric Name / Indicator	Target / SLA
Performance	Auth Latency (p99)	< 10 ms (Hit Path)

Metric Category	Metric Name / Indicator	Target / SLA
Cache Efficiency	Redis Cache Hit Ratio	> 99.5% for active sessions
Security SLA	Revocation Propagation Delay	< 1 second from role change to Redis key deletion
Delegation Safety	Confused Deputy Rejections	100% Rejection (`403 Forbidden`) on invalid `aud`

Distributed Tracing: Pass OpenTelemetry / AWS X-Ray context from API Gateway through the Auth Lambda across VPC Lattice to downstream Workspace microservices.

Audit Logging: Immutable log stream capturing authorization decisions (`Allow` / `Deny`), actor (`sub`), delegate (`azp`), and target workspace.

6. Failure Modes, Tradeoffs & Mitigations

Cache Fallback Spike (Redis Unavailability):

Impact: If primary Redis cluster becomes unavailable, falling back directly to AVP under heavy load could degrade overall API latency and exhaust AVP quotas.

Mitigation: Introduce an L2 Cache layer (e.g., DynamoDB with DAX) or partition/shard the primary Redis cache across distinct tenants, sectors, or geographic regions to limit blast radius during cache outages.

Compute Cost on Malformed Tokens:

Tradeoff: Because JWT signature verification runs inside the custom Lambda Authorizer rather than natively at the Gateway level, malformed or forged tokens consume Lambda execution time before rejection.

In-Memory JWKS Management:

Mitigation: The Auth Lambda caches `jwks.json` in container memory and performs zero-allocation string parsing, refreshing keys only when encountering an unknown `kid` header.

7. FinOps & Future Evolution (ADR 2.0 Roadmap)

Cost Baseline

The managed architecture (ADR 001) costs approximately **~$2,000 per 1 Billion requests** under healthy Redis cache hit rates (> 99.5%).

Migration Triggers to Self-Hosted Architecture (ADR 2.0)

We will continuously monitor infrastructure unit costs and traffic growth. A transition to a self-hosted authorization architecture (e.g., containerized Envoy Proxy with embedded Cedar Rust engine and self-hosted Ory/Keycloak identity) will be planned when:

1. **Engineering Capacity**: Platform engineering team allocates dedicated operational support to maintain, patch, and manage SLAs for containerized auth infrastructure.
2. **Cost Optimization ROI**: Throughput scaling makes the **80% to 90% cost reduction** (dropping monthly operational costs from ~$2,000 to ~$200–$300 per 1B requests) high ROI relative to team maintenance overhead.

Sources:

https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html

https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html